

Simple Calculator - Python Implementation

Introduction

This guide will walk you through creating a basic calculator using Python. It is designed for beginners with little or no programming experience. By the end of this guide, you will have a simple calculator that can perform addition, subtraction, multiplication, division, square root, cube root, and modulus calculations.

Overview

- The Simple Calculator project is a command-line application that allows users to:
- Perform addition, subtraction, multiplication, division, and modulus operations.
- Continuously perform calculations until the user decides to exit.
- Handle invalid input and prevent division by zero errors.
- Display meaningful error messages when the user enters incorrect inputs.

Prerequisites

Before starting, ensure you have:

- Python installed on your computer (Download from [Python's official website](#))
- A code editor (such as **Colab**, **VS Code**, or **PyCharm**)
- Basic knowledge of mathematical operations
- Basic Python syntax.
- Functions and conditional statements.
- Loops for repeated execution.

Why Build a Calculator?

Building a calculator is an excellent beginner project for learning:

- Basic arithmetic operations
- Functions and modular programming
- User input handling
- Looping mechanisms to keep the program running until the user exits

Features to Implement

Before we start coding, let's define the key features our calculator should have:

1. **User Input Handling** – Allow users to enter numbers and select operations.
2. **Basic Operations** – Perform addition, subtraction, multiplication, division, and remainder (modulus).
3. **Validation Mechanism** – Ensure correct numeric input without using exception handling.
4. **Looping Mechanism** – Enable multiple calculations without restarting the program.
5. **User-Friendly Interface** – Display clear instructions and formatted output.

Project is Built Using:

- **Python:** The programming language used for implementing the calculator.
 - **math module:** A built-in Python module used for advanced mathematical operations, such as calculating square roots.
-

Setting Up the Project

First, Create a new project folder and add the following file structure:

```
/simple-calculator
|— calculator.py (Main Python script)
```

Writing the Python Code

Open **calculator.py** and add the following code:

Functions and Their Descriptions

1. Addition Function

```
def add(x, y):
    return x + y
```

- This function takes two numerical inputs and returns their sum

2. Subtraction Function

```
def subtract(x, y):  
    return x - y
```

- This function takes two numbers and returns their difference

3. Multiplication Function

```
def multiply(x, y):  
    return x * y
```

- function takes two numbers and returns their product.

4. Division Function

```
def divide(x, y):  
    if y != 0:  
        return x / y  
    else:  
        return "Error! Division by zero."
```

- This function divides the first number by the second.
- If the denominator is zero, an error message is returned to prevent division error

5. Modulus Function

```
def modulus(x, y):  
    return x % y
```

- Returns the remainder when the first number is divided by the second.

Switch Case Function

```
def switch_case(operation, num1, num2=None):
    match operation:
        case '1':
            return add(num1, num2)
        case '2':
            return subtract(num1, num2)
        case '3':
            return multiply(num1, num2)
        case '4':
            return divide(num1, num2)
        case '5':
            return modulus(num1, num2)
        case _:
            return "Invalid input. Please enter a valid number from the menu."
```

- This function matches the operation chosen by the user and calls the corresponding function to perform the calculation.

Running the Calculator

```
print("Simple Calculator")
print("Select operation:")
print("1. Add")
print("2. Subtract")
print("3. Multiply")
print("4. Divide")
print("5. Modulus")

while True:
    choice = input("Enter choice (1/2/3/4/5): ")

    if choice in ['1', '2', '3', '4', '5']:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = switch_case(choice, num1, num2)
```

```
else:
    print("Invalid input. Please enter a valid number from the menu.")
    continue

print(f"Result: {result}")

while True:
    next_calculation = input("Do you want to perform another
calculation? (yes/no): ").lower()
    if next_calculation in ['yes', 'no']:
        break
    print("Please provide 'yes' or 'no' to continue or exit the
calculation.")

if next_calculation == 'no':
    print("Goodbye! Thank you for using the calculator")
    break
```

Display Operations and Input Selection:

- The code first prints a menu with available operations (**Add, Subtract, Multiply, Divide, Modulus**). The program then prompts the user to choose an operation (1-5). If the user selects a valid option, the program proceeds to request two numbers for the calculation.

Performing the Calculation:

- Based on the selected operation, the program calls the appropriate function (**switch_case**) to perform the desired arithmetic operation on the two numbers provided by the user. It then prints the result.

Allowing Multiple Calculations:

- After each calculation, the program asks if the user wants to perform another calculation. If the user types **'yes'**, the loop repeats, prompting for new inputs. If **'no'**, the program exits the loop and ends. If the user enters invalid responses, it asks again until a valid input is provided.

Example Usage

Example 1: Addition

Enter choice (1/2/3/4/5): 1

Enter first number: 10

Enter second number: 5

Result: 15.0

Do you want to perform another calculation? (yes/no): no

How to Run the Project

1. Install Python (if not installed).
 2. Open the terminal or command prompt.
 3. Navigate to the project folder.
 4. Run the script using:
python calculator.py
 5. Follow the on-screen menu to perform calculations.
-

What You Learned:

- How to structure a Python program using functions.
 - How to handle user input and prevent errors.
 - How to use loops to improve user experience.
 - How to validate input to ensure correct calculations.
-

Conclusion

This documentation explains how the Simple Calculator project works, covering its functions, logic, and user interactions. This project helps beginners understand Python fundamentals while providing practical hands-on experience in building an interactive application.

Refer to this [Link](#) for project

Happy Coding!